# Chapter 4 - Loops

## Why Do We Need Loops?

A loop repeats a block of code many times, so you do not have to write the same lines again and again. Python has two kinds of loops: the for loop and the while loop.

## What is a for Loop?

A for loop iterates over every item of a sequence such as a list, tuple, string, dictionary or the numbers produced by range(). On each repetition the loop variable takes the next item. Use a for loop when you know in advance what you want to loop over.

```
for fruit in ["apple", "banana", "mango"]:
    print(fruit)

for i in range(1, 6):      # 1, 2, 3, 4, 5
    print(i)
```

range(start, stop, step) generates numbers from start up to but not including stop. Looping through a dictionary with .items() gives both the key and the value, and enumerate() gives the index and the item together.

```
marks = {"Ali": 80, "Sara": 92}
for name, score in marks.items():
    print(name, score)

for index, fruit in enumerate(["apple", "banana"]):
    print(index, fruit)
```

## What is a while Loop?

A while loop keeps running as long as its condition stays True. Use it when you do not know in advance how many repetitions are needed. Make sure something inside the loop eventually makes the condition False, otherwise you create an infinite loop.

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

## break, continue and else

- break: stops the loop immediately.
- continue: skips the rest of the current repetition and goes to the next one.
- else: an optional block that runs when the loop finishes without hitting break.

## Nested Loops and List Comprehensions

A loop inside another loop is called a nested loop; it is often used for tables and grids. A list comprehension is a short way to build a list with a loop in a single line.

```
squares = [n * n for n in range(1, 6)]   # [1, 4, 9, 16, 25]
evens = [n for n in range(10) if n % 2 == 0]
```